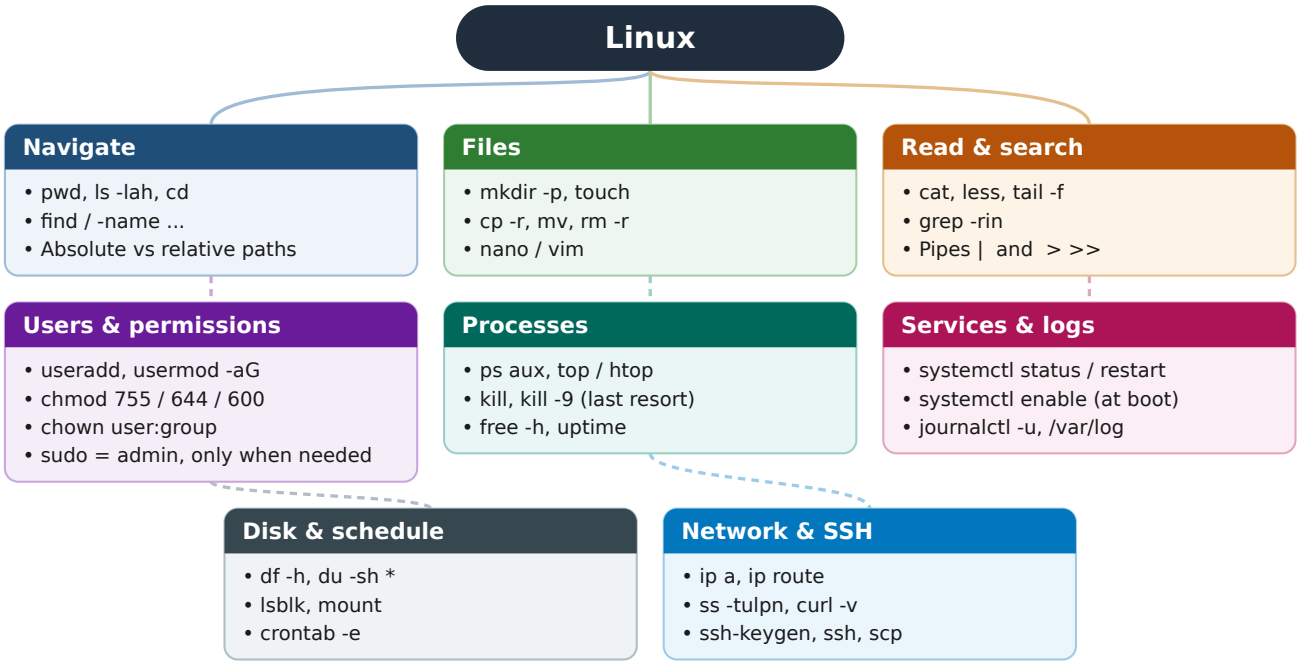
3. Linux

DevOps, Cloud & Platform Engineering Notes · Section 3 of 18 · Thirupathi Keloth · v1.4

In this section: 3.1 Architecture · 3.2 Reading a command · 3.3 File system layout · 3.4 Files and directories · 3.5 Viewing and searching · 3.6 Pipes and redirection · 3.7 Text processing · 3.8 Users and groups · 3.9 Permissions · 3.10 Processes, CPU and memory · 3.11 Package management (Ubuntu) · 3.12 Services and systemd · 3.13 Environment variables and shell config · 3.14 Linux networking · 3.15 SSH · 3.16 Logs · 3.17 Disks and storage · 3.18 cron (scheduled jobs) · 3.19 Linux security basics · 3.20 Linux troubleshooting method · 3.21 More essential commands · 3.22 Growing a disk (real-world task, e.g. after enlarging an AWS EBS volume)



3.1 Architecture

```
Applications  (nginx, python, docker)
     |
Shell  (bash: reads your commands)
     |
Kernel  (manages CPU, memory, disks, network, processes)
     |
Hardware
```

- **Distribution (distro):** the kernel plus tools, packaged together. Ubuntu/Debian use `apt`. RHEL/Amazon Linux/Rocky use `dnf` / `yum`.
- **Everything is a file:** devices, settings and processes all appear as files.

3.2 Reading a command

```
ls   -l   -a   /etc
|     └─options─┘   |
command            argument (what to act on)
```

- Options can be combined: `ls -la`. Long form: `--all`.
- Help: `man ls`, `ls --help`. Quit `man` with `q`.
- Shortcuts: `Tab` = autocomplete, `↑` = previous command, `Ctrl+C` = stop, `Ctrl+L` = clear, `history` = past commands.

3.3 File system layout

Directory	Contains
`/`	Root: top of everything
`/home/<user>`	Users' personal files (`~` = your home)
`/root`	Home of the root (admin) user
`/etc`	Configuration files
`/var`	Changing data: logs (`/var/log`), web files, caches
`/bin` , `/usr/bin`	Programs (commands)
`/tmp`	Temporary files, cleared on reboot
`/opt`	Optional / third-party software
`/dev`	Devices (disks, terminals)
`/proc`	Live kernel and process information
`/mnt` , `/media`	Mount points for extra disks

Absolute path: starts from `/` (`/etc/nginx/nginx.conf`). **Relative path:** starts from where you are (`./logs` , `../config`). `.` = here, `..` = one level up.

3.4 Files and directories

Command	What it does
`pwd`	Print the current directory
`ls -lah`	List all files, long format, human-readable sizes
`cd /path` , `cd ~` , `cd ..` , `cd -`	Change directory: path, home, up, previous
`mkdir -p a/b/c`	Create directories (with parents)
`touch file`	Create an empty file / update its timestamp
`cp src dst` , `cp -r dir1 dir2`	Copy a file / a directory
`mv old new`	Move or rename
`rm file` , `rm -r dir`	Delete. **No recycle bin.** Be careful with `rm -rf`
`ln -s target link`	Create a shortcut (symbolic link)
`file name`	Show the file type
`tree`	Show a directory tree

Wildcards: `*` = anything, `?` = one character. Example: `ls *.log`.

3.5 Viewing and searching

Command	What it does
`cat file`	Print the whole file
`less file`	Scroll through a file (`/word` to search, `q` to quit)
`head -n 20 file` , `tail -n 20 file`	First / last 20 lines
`tail -f /var/log/syslog`	Follow a log live
`grep "error" file`	Find lines containing the text
`grep -i -r -n "error" /var/log`	Case-insensitive, recursive, with line numbers
`grep -v "debug"`	Lines that do **not** match
`find / -name "*.conf"`	Find files by name
`find /var -size +100M`	Find files bigger than 100 MB
`find . -mtime -1`	Files changed in the last day

3.6 Pipes and redirection

Symbol	Meaning	Example
`|`	Send the output of one command into another	`cat log | grep error`
`>`	Write output to a file (overwrite)	`ls > list.txt`
`>>`	Append output to a file	`echo hi >> notes.txt`
`2>`	Redirect errors	`cmd 2> errors.txt`
`&>` / `2>&1`	Redirect output and errors together	`cmd > all.txt 2>&1`
`<`	Use a file as input	`wc -l < file`

Streams: **stdin** (0) = input, **stdout** (1) = normal output, **stderr** (2) = errors.

3.7 Text processing

Command	What it does	Example
`wc -l`	Count lines	`wc -l access.log`
`sort` , `sort -n` , `sort -r`	Sort lines (numeric, reverse)	`sort names.txt`
`uniq -c`	Count repeated lines (input must be sorted)	`sort ips | uniq -c`
`cut -d',' -f2`	Pick a column	`cut -d':' -f1 /etc/passwd`
`awk '{print $1}'`	Print a field	`awk '{print $1}' access.log`
`sed 's/old/new/g'`	Find and replace	`sed -i 's/8080/80/g' app.conf`
`tr` , `xargs`	Translate characters / build commands from input	`cat files | xargs rm`

Top 5 IPs in a web log: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`

Editors: `nano file` (simple: `Ctrl+O` save, `Ctrl+X` exit). `vim file` (`i` = insert mode, `Esc` = command mode, `:wq` = save and quit, `:q!` = quit without saving).

Archives: `tar -czvf backup.tar.gz dir/` (create), `tar -xzvf backup.tar.gz` (extract), `zip` / `unzip`.

3.8 Users and groups

Command / file	What it does
`whoami`, `id`	Current user, its UID and groups
`sudo command`	Run one command as root (admin)
`sudo useradd -m -s /bin/bash alice`	Create a user with a home directory
`sudo passwd alice`	Set a password
`sudo usermod -aG devops alice`	Add a user to a group (`-a` = append, don't forget it)
`sudo groupadd devops`	Create a group
`sudo userdel -r alice`	Delete a user and their home directory
`su - alice`	Switch to another user
`/etc/passwd`, `/etc/group`, `/etc/shadow`	Users, groups, password hashes
`sudo visudo`	Safely edit sudo permissions

Best practice: never work as root day to day. Use `sudo` only when needed (least privilege).

3.9 Permissions

```
-rwxr-x---  1  alice  devops  4096  app.sh
│└┘└┘└┘      owner  group
│ │  │  └ others: ---  (nothing)
│ │  └ group:  r-x  (read, execute)
│ └ owner:     rwx  (read, write, execute)
└ type: - file, d directory, l link
```

Letter	Number	On a file	On a directory
r	4	Read content	List files
w	2	Change content	Create/delete files inside
x	1	Run as a program	Enter (`cd`) it

Command	Meaning
`chmod 755 file`	rwx r-x r-x (scripts, directories)
`chmod 644 file`	rw- r-- r-- (normal files)
`chmod 600 key.pem`	rw- --- --- (private keys, secrets)
`chmod u+x script.sh`	Add execute for the owner
`chown alice:devops file`	Change owner and group

Command	Meaning
`chown -R` / `chmod -R`	Apply recursively
`umask`	Default permissions for new files

Special bits: setuid (runs as the file owner, e.g. `passwd`), setgid (new files inherit the group), sticky bit (only the owner can delete, e.g. `/tmp`).

3.10 Processes, CPU and memory

Command	What it does
`ps aux`	All running processes
`ps aux | grep nginx`	Find a process
`top` / `htop`	Live CPU and memory view (`q` to quit; in top, `P` = sort by CPU, `M` = by memory)
`kill PID`	Ask a process to stop (SIGTERM, 15)
`kill -9 PID`	Force-kill (SIGKILL). Last resort
`pkill name`	Kill by name
`command &`, `jobs`, `fg`, `bg`	Run in the background and manage jobs
`nohup command &`	Keep running after you log out
`nice -n 10 cmd`, `renice`	Lower / change priority
`free -h`	RAM and swap usage
`uptime`	Load average for 1, 5 and 15 minutes
`nproc`	Number of CPU cores
`vmstat 1`, `iostat`	CPU, memory and disk statistics

Load average: compare it with the number of cores. Load 4.0 on 4 cores = fully busy. **Swap** = disk used as extra RAM; heavy swapping = slow.

3.11 Package management (Ubuntu)

Command	What it does
`sudo apt update`	Refresh the list of available packages
`sudo apt upgrade`	Install updates
`sudo apt install nginx`	Install a package
`sudo apt remove nginx` / `purge`	Remove (purge also deletes config)
`apt search name`, `apt show name`	Search / show details
`dpkg -l`	List installed packages

RHEL / Amazon Linux equivalent: `sudo dnf install nginx`.

3.12 Services and systemd

systemd starts and manages services (background programs called daemons).

Command	What it does
`systemctl status nginx`	Is it running? Shows recent logs
`sudo systemctl start / stop / restart nginx`	Control the service now
`sudo systemctl reload nginx`	Re-read config without stopping
`sudo systemctl enable nginx`	Start automatically at boot
`sudo systemctl disable nginx`	Don't start at boot
`systemctl list-units --type=service`	List services
`sudo systemctl daemon-reload`	Reload after editing unit files

Simple unit file `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My App
After=network.target

[Service]
User=appuser
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
EnvironmentFile=/etc/myapp/env

[Install]
WantedBy=multi-user.target
```

3.13 Environment variables and shell config

Command	What it does
`env` / `printenv`	List environment variables
`echo $HOME`, `echo $PATH`	Show a variable
`export APP_ENV=dev`	Set a variable for this session and child processes
`unset APP_ENV`	Remove it
`~/.bashrc`	Runs at every new shell: aliases, exports
`source ~/.bashrc`	Reload it now
`alias ll='ls -lah'`	Create a shortcut
`which python3`	Where a command lives (searched via `PATH`)

Security: never hardcode secrets in scripts or code. Load them from environment variables or a secrets manager, and never commit `.env` files.

3.14 Linux networking

Command	What it does
`ip a`	Interfaces and IP addresses
`ip route`	Routing table (default gateway)
`ss -tulpn`	Listening ports and the program using each one
`ping`, `traceroute`, `mtr`	Reachability and path
`dig name`, `nslookup name`	DNS lookup
`curl -I https://site`	HTTP headers only
`curl -v http://localhost:8080`	Full request/response details
`wget url`	Download a file
`nc -zv host 22`	Test a TCP port
`sudo tcpdump -i eth0 port 80`	Capture packets
`/etc/hosts`, `/etc/resolv.conf`	Local name overrides / DNS servers in use

nginx reverse proxy and load balancer:

```
upstream backend {
    server 127.0.0.1:5001;
    server 127.0.0.1:5002;
}
server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}
```

Test the config with `sudo nginx -t`, then `sudo systemctl reload nginx`.

3.15 SSH

Command	What it does
`ssh-keygen -t ed25519 -C "me@laptop"`	Create a key pair (`~/.ssh/id_ed25519` and `.pub`)
`ssh user@host`	Connect
`ssh -i key.pem ubuntu@1.2.3.4`	Connect with a specific key
`ssh-copy-id user@host`	Install your public key on a server
`scp file user@host:/tmp/`	Copy a file to a server
`rsync -avz dir/ user@host:/backup/`	Efficient sync / copy
`ssh -L 8080:localhost:80 user@host`	Tunnel: local port 8080 → server's port 80

`~/.ssh/config` shortcut:

```
Host web1
    HostName 1.2.3.4
    User ubuntu
    IdentityFile ~/.ssh/web1.pem
```

Hardening (`/etc/ssh/sshd_config`): `PasswordAuthentication no` , `PermitRootLogin no` , keys only, private key `chmod 600` , then `sudo systemctl restart ssh` .

3.16 Logs

Command / path	What it does
`/var/log/syslog` (Ubuntu), `/var/log/messages` (RHEL)	General system log
`/var/log/auth.log`	Logins, sudo, SSH attempts
`/var/log/nginx/access.log` , `error.log`	Web server logs
`journalctl -u nginx`	Logs of one service
`journalctl -u nginx -f`	Follow live
`journalctl --since "1 hour ago"`	Recent logs
`journalctl -p err -b`	Errors since the last boot
`dmesg`	Kernel messages (hardware, out-of-memory kills)
`logrotate`	Rotates and compresses old logs so disks don't fill

3.17 Disks and storage

Command	What it does
`df -h`	Free space per filesystem
`df -i`	Free inodes (a disk can be "full" of tiny files)
`du -sh *`	Size of each item in the current directory
`du -h / --max-depth=1 2>/dev/null | sort -h`	Find the biggest top-level directories
`lsblk`	Disks and partitions
`sudo mkfs.ext4 /dev/xvdf`	Create a filesystem (**erases the disk**)
`sudo mount /dev/xvdf /data`	Attach a filesystem to a directory
`/etc/fstab`	Mounts done automatically at boot

Disk full checklist: `df -h` → `du` to find big directories → old logs (`/var/log`), journal (`journalctl --vacuum-size=200M`), caches, old Docker images (`docker system prune`). Deleted but still-open files: `lsof +L1` .

3.18 cron (scheduled jobs)

```
*  *  *  *  *   command
|  |  |  |  └ day of week (0-7, Sun = 0 or 7)
|  |  |  └─── month (1-12)
|  |  └────── day of month (1-31)
|  └───────── hour (0-23)
└──────────── minute (0-59)
```

Example	Runs
`0 2 * * * /opt/backup.sh`	Every day at 02:00
`*/5 * * * * /opt/check.sh`	Every 5 minutes
`0 9 * * 1-5`	09:00, Monday to Friday

`crontab -e` (edit), `crontab -l` (list). Always use full paths and redirect output: `>> /var/log/backup.log 2>&1`.

3.19 Linux security basics

- Keep the system updated: `sudo apt update && sudo apt upgrade`.
- Firewall (`ufw`): `sudo ufw allow 22/tcp`, `sudo ufw allow 80`, `sudo ufw enable`, `sudo ufw status`.
- SSH keys only, no root login, consider `fail2ban` to block brute-force attempts.
- Least privilege: separate users, minimal sudo, correct permissions.
- Remove unused packages and services. Check listening ports with `ss -tulpn`.
- Review `/var/log/auth.log` for suspicious logins.

3.20 Linux troubleshooting method

1. **What exactly is wrong?** Error message, since when, what changed?
2. **Is the service running?** `systemctl status`
3. **What do the logs say?** `journalctl -u`, `/var/log/...`
4. **Resources:** `top`, `free -h`, `df -h`
5. **Network:** `ss -tulpn`, `curl localhost`, firewall rules
6. **Config and permissions:** `nginx -t`, `ls -l`
7. Fix one thing at a time, then re-test. Write down what you did.

3.21 More essential commands

Command	What it does
`lsof -i :80`	Which process is using port 80
`lsof +L1`	Deleted files still held open (disk "full" but nothing visible)
`hostnamectl`, `timedatectl`	Show/set hostname / time zone and NTP sync
`uname -a`, `cat /etc/os-release`	Kernel version / Linux distribution
`systemctl --failed`	Services that failed
`dmesg -T | grep -i oom`	Processes killed by the out-of-memory (OOM) killer
`watch -n 2 'df -h'`	Re-run a command every 2 seconds

Command	What it does
`stat file`	Detailed file info (size, permissions, timestamps)
`sha256sum file`	Checksum to verify a download
`ulimit -n`	Max open files for the shell (a common limit on busy servers)

3.22 Growing a disk (real-world task, e.g. after enlarging an AWS EBS volume)

```
lsblk                                  # see the bigger disk and the old partition size
sudo growpart /dev/nvme0n1 1           # grow partition 1 to fill the disk
sudo resize2fs /dev/nvme0n1p1          # grow an ext4 filesystem
sudo xfs_growfs /                      # or: grow an XFS filesystem (Amazon Linux default)
df -h                                  # confirm the new size
```

LVM (Logical Volume Manager): combines disks into flexible volumes. Physical volume (`pvcreate`) → volume group (`vgcreate`) → logical volume (`lvcreate`); grow with `lvextend -r` . **Swap:** extra "memory" on disk. Check with `swapon --show` . Useful on small servers, but heavy swapping means you need more RAM. **SELinux / AppArmor:** extra security layers (RHEL / Ubuntu) that can block a service even when file permissions look right. Check with `getenforce` or `aa-status` .

REMEMBER

- Permissions: **r = 4, w = 2, x = 1** → 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--. Private keys: **600**.
- Cron fields in order: **minute hour day month weekday** ("**M**y **H**amster **D**oesn't **M**ind **W**aiting").
- Service broken? **status → logs → resources → network → config**.
- Linux has no recycle bin: double-check before `rm -rf` .